



SHELL CHEAT SHEET

[SKILLS: 26 • SECTIONS: 6]

Shell scripting is crucial for system administration and automation. This Skill Tree offers a systematic way to master command - line interfaces and shell programming. Tailored for beginners, it provides a clear path to understand Unix/Linux shells and scripting languages. Hands - on, non - video courses and practical exercises in an interactive command - line environment help you develop real - world shell scripting skills.

TABLE OF CONTENTS

[SECTIONS: 6 • COMMANDS: 26]

- 1. BASIC SYNTAX AND STRUCTURE
3 items
- 2. VARIABLE HANDLING
5 items
- 3. CONTROL FLOW
7 items
- 4. FUNCTIONS AND SCOPE
1 items
- 5. ADVANCED SCRIPTING CONCEPTS
6 items
- 6. SYSTEM INTERACTION AND CONFIGURATION
4 items

1. BASIC SYNTAX AND STRUCTURE

Basic syntax and structure elements in shell scripting, including Shebang, Comments, and Quoting Mechanisms.

- **Shebang** 01
The shebang is a special line at the beginning of a shell script that specifies the interpreter to be used for executing the script.
- **Comments** 02
Comments in shell scripting are used to add explanations or documentation within the script. They are ignored by the shell and serve as notes for human readers.
- **Quoting Mechanisms** 03
Quoting mechanisms in shell scripting are used to control the interpretation of special characters within strings or variables. Common types include single quotes, double quotes, and backslashes.

4. FUNCTIONS AND SCOPE

Topics related to defining functions, understanding variable scope, and managing data within shell scripts.

- **Function Definition** 01
Function definition in shell scripting allows you to define reusable blocks of code with their own local variables and parameters.

2. VARIABLE HANDLING

Techniques for declaring, using, and manipulating variables in shell scripting, including String Manipulation, Arrays, Associative Arrays, Parameter Expansion, and Exit Status Checks.

- . Variable Declaration 01
Variable declaration involves creating and initializing variables to store data. In shell scripting, variables are typically assigned values using the '=' operator.
- . Variable Usage 02
Variable usage refers to the process of accessing and using the values stored in variables within a shell script.
- . String Manipulation 03
String manipulation involves operations on string variables, such as concatenation, substring extraction, and string length.
- . Arrays 04
Arrays in shell scripting allow you to store multiple values in a single variable. You can access individual elements using indices.
- . Parameter Expansion 05
Parameter expansion is a technique for modifying and expanding the values of variables using special operators and patterns.

5. ADVANCED SCRIPTING CONCEPTS

Advanced concepts in shell scripting, including Arithmetic Operations, Arithmetic Expansion, Reading Input, Command Substitution, Subshells and Command Groups, Advanced Redirection, and Here Strings.

- . Arithmetic Operations 01
Arithmetic operations in shell scripting allow you to perform mathematical calculations using variables and constants.
- . Arithmetic Expansion 02
Arithmetic expansion involves evaluating mathematical expressions within double parentheses to obtain numeric results.
- . Reading Input 03
Reading input in shell scripts involves methods for capturing user or file input and using it in the script's logic.
- . Command Substitution 04
Command substitution allows you to capture the output of a command and use it as part of a shell script's command or expression.
- . Subshells and Command Groups 05
Subshells and command groups are used to execute commands within a subshell environment or as a group, affecting variable scope and execution context.
- . Advanced Redirection 06
Advanced redirection techniques in shell scripting involve directing input and output streams to and from files or other processes.

3. CONTROL FLOW

Control flow structures in shell scripting are used to manage the execution flow of a script. They include If-Else Statements, Case Statements, For Loops, While Loops, Until Loops, Conditional Expressions, Exit and Return Status.

- **If-Else Statements** 01
If-Else Statements in shell scripting allow you to make decisions based on conditions. You can specify actions to be taken when a condition is true (if) or false (else).
- **Case Statements** 02
Case Statements provide a way to perform different actions based on the value of a variable or expression. They are useful for handling multiple conditions.
- **For Loops** 03
For Loops in shell scripting are used to iterate over a list of items and execute a set of commands for each item in the list.
- **While Loops** 04
While Loops allow you to repeatedly execute a block of commands as long as a specified condition is true.
- **Until Loops** 05
Until Loops are similar to While Loops but execute a block of commands until a specified condition becomes true.
- **Conditional Expressions** 06
Conditional expressions are used to evaluate conditions and return a true or false value, which is often used in If-Else and While Loops.
- **Exit and Return Status** 07
Exit and Return Status refer to the exit code or status returned by a shell script when it finishes executing. It can be used to indicate success or failure.

6. SYSTEM INTERACTION AND CONFIGURATION

Shell scripting features related to system interaction, configuration, and control, including Trap Statements, Signal Handling and Job Control, Shell Options and Attributes, Globbing and Pathname Expansion.

- **Exit Status Checks** 01
Exit status checks involve evaluating the return status of commands or scripts to determine success or failure.
- **Trap Statements** 02
Trap statements in shell scripting are used to set up signal handlers that specify how the script should respond to various signals, such as interrupts or errors.
- **Shell Options and Attributes** 03
Shell options and attributes allow you to configure the behavior and characteristics of the shell environment.
- **Globbing and Pathname Expansion** 04
Globbing and pathname expansion are techniques for matching and expanding file and directory paths using wildcard characters and patterns.